P2.4. Guion del profesorado

Diagnóstico: herramientas del navegador y tipos de error

??ndice

Ficha de la actividad **1**

Nota sobre la clasificación en tres tipos **2**

Los seis fallos de la página **2**
- Comprobaciones hechas sobre los ficheros . . . 2
- Orden en que suelen aparecer . . . 3

Clave del ejercicio de clasificación **3**
- El caso 6, que da discusión . . . 3
- La pregunta final . . . 4

Rúbrica analítica **5**
- C1. Localización de los fallos . . . 5
- C2. Uso de las herramientas del navegador . . . 5
- C3. Clasificación de los errores . . . 5
- C4. Cambios en caliente y su interpretación . . . 5
- C5. Entrega y evidencias . . . 6

Reparto del tiempo **6**

Dificultades previstas **6**

Ficha de la actividad

Unidad	UD2. Instalación y uso de entornos de desarrollo
Duración estimada	1 sesión de 2 h, o 2 sesiones de 1 h (Parte 3 / Parte 4)
Modalidad	Individual, en el aula, con ordenador
Material	`P2.4-ficheros.zip`, navegador con herramientas de desarrollo
Requisitos previos	VS Code con Live Server (documento de VS Code con HTML)
Entregable	`P24_APELLIDOS.zip`: proyecto corregido, tabla de diagnóstico, capturas

Qué evalúa de verdad. No evalúa saber HTML ni Python: evalúa **localizar un fallo con la herramienta adecuada y nombrarlo correctamente**. Por eso la columna «Dónde lo detecté» de la tabla del alumnado vale tanto como la corrección.

Nota sobre la clasificación en tres tipos

El documento del alumnado usa **tres** categorías —sintaxis, ejecución y significado— en lugar de las dos habituales.

El motivo es que con solo dos no hay dónde meter un `TypeError`: el programa está bien escrito (no es de sintaxis) y hace lo que el código dice (no es de significado), pero se para. Si se obliga a elegir entre dos, la mitad de la clase lo llama de sintaxis porque «sale un error rojo», que es justo la confusión que interesa evitar.

Con tres categorías la frontera queda limpia y operativa:

Pregunta que se hace el alumno	Respuesta	Tipo
¿Ha llegado a arrancar?	No	Sintaxis
¿Ha terminado?	No	Ejecución
¿El resultado es el pedido?	No	Significado

Si en la programación del módulo se manejan las cinco clásicas (léxico, sintáctico, semántico, lógico y de ejecución), la equivalencia es: *sintaxis* agrupa léxico y sintáctico, *significado* agrupa semántico y lógico.

Los seis fallos de la página

Van numerados igual que en el enunciado del alumnado.

Nº	Dónde	Qué está mal	Corrección	Tipo	Se detecta en
1	`index.html`, línea del `<link>`	`href="estilos.css"` y el fichero está en `css/`	`href="css/estilos.css"`	Ejecución (404)	Consola, Red
2	`index.html`, el `<img>`	`src="imagenes/avatar.png"` y la carpeta es `img/`	`src="img/avatar.png"`	Ejecución (404)	Consola, Red
3	`index.html`, última fila de la tabla	Falta `</td>` y `</tr>`	Cerrar las dos etiquetas	Sintaxis	Elementos
4	`index.html`, segunda `<section>`	`id="datos"` repetido	Cambiarlo por `id="herramientas"`	Sintaxis	VS Code, Elementos
5	`css/estilos.css`	`h1 { color: #c0392b }` queda pisada por `header h1`	Quitar la regla suelta, o dejarla si se asume	Significado	Estilos
6	`index.html`, primer enlace	`href=""`	`href="https://github.com/tuusuario/tienda"`	Significado	Elementos

Comprobaciones hechas sobre los ficheros

Los fallos están verificados ejecutando la página en un navegador real:

- Se producen exactamente **dos** peticiones 404: `estilos.css` y `imagenes/avatar.png`.
- El navegador **cierra solo** la fila incompleta: el DOM muestra `</td></tr>` que no están en el fichero. Es el ejemplo que sostiene toda la Parte 1.3.

- `document.querySelectorAll('[id="datos"]')` devuelve **2** elementos.
- Con el CSS sin cargar, el <h1> sale negro; al corregir el fallo 1, pasa a azul #1F4E79 y la regla roja aparece **tachada**. Es decir, **el fallo 5 no se puede ver hasta arreglar el 1**: conviene decirlo si alguien se atasca buscándolo antes de tiempo.

Orden en que suelen aparecer

Los fallos 1 y 2 salen en los primeros treinta segundos. El 3 y el 5 requieren abrir paneles y son los que separan. El 4 se encuentra buscando en el fichero, no en el navegador, y el 6 solo aparece si alguien prueba los enlaces: **avisar de que hay que probarlos**, porque pasar el ratón por encima no basta.

Clave del ejercicio de clasificación

Caso	Tipo	Qué se veía	Quién lo detecta
1	Sintaxis	`SyntaxError: expected ':'`. No arranca	El editor, subrayado, antes de ejecutar
2	Significado	Arranca y termina. Imprime `Media: 3.0` en vez de `7.0`	Solo quien compruebe el resultado
3	Ejecución	`TypeError: can only concatenate str (not "float") to str`. Se para	El propio programa al llegar ahí
4	Significado	Para 18 años contesta `False`	Solo quien pruebe el caso límite
5	Sintaxis	La lista se ve, pero el árbol no coincide con el fichero	HTMLHint y la pestaña Elementos
6	Sintaxis	Funciona por casualidad; falla en cuanto el nombre lleve un espacio	HTMLHint
7	Significado	El enlace funciona perfectamente y lleva a otro sitio	Solo quien lo pulse
8	Significado	La tabla se ve bien, con los datos cambiados de columna	Solo quien lea el contenido

Salidas reales comprobadas de los casos de Python, por si se proyectan en clase:

```
Caso 1 → SyntaxError: expected ':'  (línea 2)
Caso 2 → Media: 3.0
Caso 3 → TypeError: can only concatenate str (not "float") to str
Caso 4 → False
```

El caso 6, que da discusión

`<img src=foto.png alt="Mi foto">` **funciona** en todos los navegadores: HTML admite atributos sin comillas si el valor no tiene espacios. Es un error de sintaxis que no se manifiesta.

Es un buen punto de parada en clase: **un error que hoy no da la cara es un error igual**, y lo dará el día que

el fichero se llame `mi foto.png`. Sirve para justificar por qué se hace caso a HTMLHint aunque la página se vea bien.

La pregunta final

«¿Cuáles habrías encontrado sin que nadie te dijera que hay un fallo?» La respuesta esperada es **1, 3 y 5** (el ordenador avisa) frente a **2, 4, 7 y 8** (no avisa nadie), con el 6 en tierra de nadie.

Lo que se busca es que escriban la conclusión: **los errores de significado solo aparecen si alguien comprueba el resultado**. Quien llegue ahí por su cuenta ha entendido la actividad entera.

Rúbrica analítica

Cinco criterios, 2 puntos cada uno.

C1. Localización de los fallos

Nivel	Descriptor
Excelente (2)	Encuentra y corrige los seis. La página queda igual que la de referencia
Sobresaliente (1,6)	Encuentra cinco; el que falta es el 5 o el 6
Bueno (1,2)	Encuentra cuatro, incluidos los dos 404
Aceptable (0,8)	Encuentra los dos 404 y alguno más
Bajo (0,4)	Solo los 404, o corrige a base de copiar la solución de otro

C2. Uso de las herramientas del navegador

Nivel	Descriptor
Excelente (2)	La columna «Dónde lo detecté» es correcta en los seis y cita la pestaña concreta
Sobresaliente (1,6)	Correcta en cinco; usa Consola, Elementos y Estilos con soltura
Bueno (1,2)	Usa Consola y Elementos; no llega al panel de Estilos
Aceptable (0,8)	Usa solo la Consola; el resto lo encuentra leyendo el código
Bajo (0,4)	No hay rastro de haber abierto las herramientas

C3. Clasificación de los errores

Nivel	Descriptor
Excelente (2)	Clasifica bien los ocho casos y los seis fallos, y justifica con el criterio correcto
Sobresaliente (1,6)	Falla uno o dos, nunca confundiendo sintaxis con significado
Bueno (1,2)	Falla tres o cuatro; distingue bien sintaxis de lo demás
Aceptable (0,8)	Clasifica a bulto: llama «de sintaxis» a todo lo que da mensaje de error
Bajo (0,4)	Tabla sin rellenar o rellenada al azar

C4. Cambios en caliente y su interpretación

Nivel	Descriptor
Excelente (2)	Las dos capturas son correctas y explica que el DOM es una copia en memoria que se rehace al recargar

Nivel	Descriptor
Sobresaliente (1,6)	Capturas correctas; explica que no se guarda, sin llegar al porqué
Bueno (1,2)	Capturas correctas; la explicación es confusa
Aceptable (0,8)	Solo una captura, o modificó el fichero en lugar de usar el panel
Bajo (0,4)	No lo ha hecho

C5. Entrega y evidencias

Nivel	Descriptor
Excelente (2)	ZIP con el nombre pedido, todo dentro, capturas legibles y con sus apellidos visibles
Sobresaliente (1,6)	Completo, con algún detalle de nombre o formato
Bueno (1,2)	Falta una evidencia
Aceptable (0,8)	Faltan varias, o las capturas no se leen
Bajo (0,4)	Entrega suelta, sin apellidos o sin proyecto

Reparto del tiempo

Minutos	Qué
0-15	Demostración en proyector: abrir F12 sobre cualquier web y recorrer las siete zonas
15-25	Parte 2 en la pizarra: los tres tipos, con los ejemplos del documento
25-35	Fase 0 y Fase 1. Puesta en común de los dos 404
35-70	Fase 2, individual
70-85	Fase 3, la tabla
85-95	Fase 4, el cambio en caliente
95-115	Parte 4, clasificación
115-120	Entrega

Si solo hay una hora, la Parte 4 se manda para casa: es la única que no necesita ordenador.

Dificultades previstas

Qué va a pasar	Cómo se corta
Abren `index.html` con doble clic en vez de Live Server	La ruta `file://` da 404 distintos y confunde. Comprobar al empezar que la barra pone `127.0.0.1:5500`
Corrigen moviendo el fichero en vez de la ruta	Vale igual, pero hay que pedir que expliquen qué hicieron
Buscan el fallo 5 antes de arreglar el 1	Recordar que sin CSS cargado no hay estilos que ver
No prueban los enlaces y no encuentran el 6	Decirlo en voz alta a mitad de la sesión
Confunden «etiqueta semántica» con «error semántico»	Está avisado en el documento; conviene decirlo también en clase

Qué va a pasar	Cómo se corta
Capturas ilegibles o recortadas	Pedir pantalla completa y comprobar los apellidos